

Utilisation de Python pour les suites et les récurrences

LPF, 2025–26 — Terminale Eds Maths — A. ABBES

On peut utiliser des boucles simples, effectuer les calculs et afficher les résultats au fur et à mesure, ou bien encore stocker les résultats dans une « liste ».

Dans ce qui suit, la librairie `math` est suffisante et peut remplacer `numpy` : `import math as np` (ou `import math as mt`, mais dans ce cas changer tous les `np` en `mt` dans le code).

1 Affichage simple

1.1 Suite récurrente

Exemple d’affichage des 50 premiers termes de la suite $u_{n+1} = 1 + \frac{1}{u_n}$, $u_0 = 1$.

```
1 import math as np
2
3 def f(x):
4     y = 1 + 1/x
5     return y
6
7 u = 1
8 for i in range(50):
9     u = f(u)
10    print(u)
```

1.2 Calcul d’une somme

Exemple de calcul de la somme $1 + \frac{1}{2} + \frac{1}{3} + \dots + \frac{1}{100}$.

```
1 n = 100
2 S = 0
3 for i in range(1, n+1):
4     S = S + 1/i
5 print(S)
```

Remarque. Cette somme devrait en fait être calculée en commençant par la fin, pour éviter des problèmes de précision lorsque n est très grand.

Exercice 1. Réécrire le programme précédent en utilisant une boucle « `while` ». (*Voir l’indication en fin de document.*)

2 Le type `list` en Python

C’est un type d’objets Python permettant de définir des suites d’objets, éventuellement de types différents entre eux, même des chaînes de caractères.

2.1 Définition et indexation

Exemple 1. Trois façons de définir la liste $l = [2, 4, 6, 8, 10]$:

Méthode	Code
Directe	<code>l = [2, 4, 6, 8, 10]</code>
Par compréhension	<code>l = [2*i for i in range(1, 6)]</code>
Par ajouts successifs	<code>l = []</code> <code>for i in range(1, 6):</code> <code> l.append(2*i)</code>

Attention. Le premier terme est indexé par 0 : ici $l[0]$ vaut 2, $l[1]$ vaut 4, etc.

2.2 Reprise de l'exemple de la section 1

```
1 import math as np
2
3 def f(x):
4     y = 1 + 1/x
5     return y
6
7 u = 1
8 l = [u]
9 for i in range(1, 50):
10     u = f(u)
11     l.append(u)
12 print(l)
```

Exercice 2. Réécrire ce code en utilisant une liste préparée à l'avance : $l = [0]*50$.
(Voir l'indication en fin de document.)

2.3 Quelques propriétés utiles

— La liste peut être parcourue directement dans une boucle `for` :

```
1 for k in l:
2     print(k)
```

— On peut en faire la somme si ce sont des nombres : `sum(l)`.

— On peut ajouter à la liste un objet de même type, ou de type différent, comme une chaîne de caractères : `l.append('nombre')`.

— Le type `list` possède beaucoup de méthodes; avec la librairie `numpy`, on dispose en plus de fonctions statistiques (moyenne, écart-type, etc.).

2.4 Tracé graphique

Remarque. Il est possible de faire un affichage graphique des valeurs de la liste l :

```
1 from matplotlib import pyplot as plt
2 plt.plot(l)
3 plt.show()
```

3 Applications

3.1 Vérification d'une formule de somme

Pour $S_n = 0 + 1 + 2 + \dots + n$, on veut vérifier que $S_n = \frac{n(n+1)}{2}$, pour n de 0 à N .

```

1 N = 20
2 S = 0
3 for n in range(N+1):
4     print(S, n*(n+1)/2)
5     S = S + n

```

Version plus concise avec des listes :

```

1 N = 20
2 L = [n*(n+1)/2 for n in range(N+1)]
3 S = [sum(range(n+1)) for n in range(N+1)]
4 print(L, S, L == S)

```

Exercice 3. Avec des listes, construire la liste des sommes cumulées et celle des valeurs $n(n+1)/2$, puis vérifier qu'elles coïncident.

```

1 S = [0]
2 L = [0]
3 for n in range(N):
4     # construire la liste des sommes et des valeurs n(n+1)/2

```

(Voir l'indication en fin de document.)

3.2 Suite récurrente $u_{n+1} = \sqrt{u_n + 1}$

Décrire cet algorithme et l'exécuter :

```

1 import math as np
2
3 u = 1
4 l = []
5 for i in range(20):
6     u = np.sqrt(u + 1)
7     l.append(u)
8 print(l)

```

Puis commenter le résultat.

Exercice 4. Programmer l'affichage des termes (et le tracé graphique) des suites des exercices des fiches 1 et 3 (récurrence). Vérifier que les 20 premiers termes vérifient bien la relation dont on demande la preuve par récurrence, puis conjecturer la convergence de ces suites.

4 Pour aller plus loin (non exigible)

4.1 Calcul symbolique avec sympy

Exemple pour les suites arithmétiques :

```

1 import sympy as sp
2
3 r, u0 = sp.symbols('r, u0')
4 u = [u0]
5 for i in range(1, 10):
6     u.append(u[i-1] + r)
7 print(u)

```

4.2 Définition récursive d'une suite

```
1 u0 = 1
2
3 def u(n):
4     if n > 0:
5         return f(u(n-1))
6     if n == 0:
7         return u0
```

5 Indications et solutions

Exercice 1 (boucle while).

```
1 n = 100
2 S = 0
3 i = 1
4 while i <= n:
5     S = S + 1/i
6     i = i + 1
7 print(S)
```

Exercice 2 (liste pré-allouée).

```
1 import math as np
2
3 def f(x):
4     return 1 + 1/x
5
6 u = 1
7 l = [0]*50
8 l[0] = u
9 for i in range(1, 50):
10     u = f(u)
11     l[i] = u
12 print(l)
```

Exercice 3 (sommes cumulées).

```
1 N = 20
2 S = [0]
3 L = [0]
4 for n in range(N):
5     S.append(S[n] + n)
6     L.append((n+1)*(n+2)/2)
7 print(S, L, S == L)
```

Exercice 4. Selon les fiches 1 et 3, adapter le modèle de la section 3.2 en remplaçant $f(x) = \text{np.sqrt}(x+1)$ par la fonction de récurrence étudiée, puis observer graphiquement la convergence.